

合肥工业大学实验预习报告

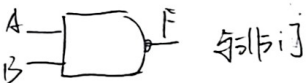
班级: 计网13-3班 学号: 1003212190 姓名: 朱清柳 日期: 2024.10.17

课程名称: 数字逻辑

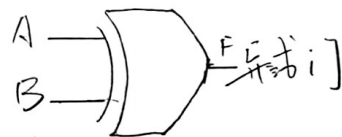
实验项目: 基本逻辑门和三态门实验

一、实验目的

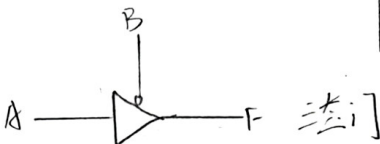
1. 掌握TTL与非门和异或门的输入与输出之间的逻辑关系。
2. 熟悉74LS14、74LS00、74LS86、74LS125、74LS04等集成芯片的外型、管脚图和使用方法。
3. 掌握三态门逻辑功能和使用方法。
4. 掌握用三态门实现数据缓冲器和数据选通的功能。
5. 学会用示波器测量信号的波形。
6. 学会测量门电路的延迟时间。



A	B	F
0	0	1
0	1	1
1	0	1
1	1	0



A	B	F
0	0	0
0	1	1
1	0	1
1	1	0



A	B	F
0	0	0
1	0	1
0	1	2
1	1	2

合肥工业大学实验报告

开课实验室: A402 成绩: 指导老师: 刘军

课程名称: 数字逻辑

实验项目: 基本逻辑门和三态门实验

一、实验仪器

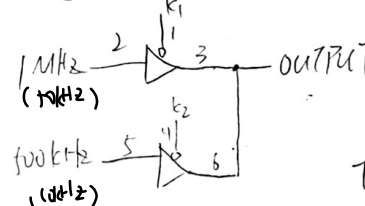
1. 二输入四与非门 74LS00
2. 二输入四异或门 74LS86
3. 三态输出的四总线缓冲器 74LS125
4. 万用表
5. 示波器

二、实验内容及步骤

1. 测试74LS00中一个与非门的输入与输出逻辑关系。
2. 测试74LS86中一个异或门的输入与输出逻辑关系。
3. 74LS125三态门输出端接为74LS00一个与非门输入端。74LS00另一个与非门输入端分别接低、高电平，测试74LS125三态门三态输出、高电平输出、低电平输出的电压值。同时测试74LS125三态门输出对74LS00的输出值。



4. 用74LS125两个三态门输出端接一异或门。使两个三态门输入均为低电平，另一个为高电平。一个三态门的输入接1MHz信号，另一个三态门的输入接500kHz信号。用示波器观察三态门的输出。



实验时因示波器问题无法显示波形，故下调频率进行实验



三、数据分析及处理

74LS00 异或门

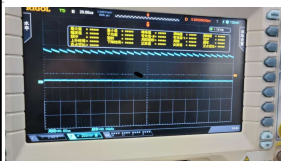
输入		输出
引脚1	引脚2	引脚3
L	L	H
L	H	L
H	L	L
H	H	L

74LS86 异或门

输入		输出
引脚1	引脚2	引脚3
L	L	L
L	H	H
H	L	H
H	H	L

输入			输出	
74LS14	74LS00	74LS14	74LS00	74LS00
引脚1	引脚2	引脚3	引脚3	引脚3
L	L	L	L	H
L	L	H	H	H
L	H	L	L	H
L	H	H	L	H
H	L	L	L	H
H	L	H	L	L
H	H	L	L	L
H	H	H	L	L

4.

输入		输出
K1	K2	
L	H	
H	L	

四、实验结果讨论

心得体会：① 学习示波器的正确使用方式

② 通过实验验证了异或门、异或门、三态门的正确输入与输出

③ 学会了如何判断元件（芯片）好坏，即验证输入与输出

④ 学会了用逻辑笔与LED灯来判断高低电平方式

实验改进不足：① 因示波器设备问题使实验步骤四中，MHz、500kHz无法正确显示频率

② 实验导线有松动现象致使示波器波形不稳定



合肥工业大学实验预习报告

班级: 计科13-3班 学号: 2013210290 姓名: 朱张初 日期: 10.22

课程名称: 数字逻辑

实验项目: 数据选择器、译码器、全加器

一、实验目的

1. 熟悉数据选择器的逻辑功能
2. 熟悉译码器的逻辑功能和使用方法
3. 设计数据选择器的电路, 进行逻辑功能的验证
4. 掌握全加器的逻辑功能

二、实验原理

74LS139 数据选择器 (4选1)

选择输入	数据输入	使能	输出
B A	C0 C1 C2 C3	G	Y
X X	X X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L
L L	L X X X	L	L

74LS139 译码器

输入	输出
G	B A
H	X X
L	L L
L	L L
L	L L
L	L L

合肥工业大学实验报告

开课实验室: A402 成绩: 指导老师: 刘军

课程名称: 数字逻辑

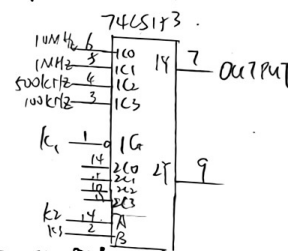
实验项目: 数据选择器、译码器、全加器

一、实验仪器

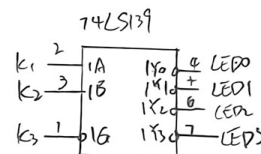
1. 双4选1数据选择器74LS139
2. 双2-4线译码器74LS139
3. 三输入NAND门74LS10
4. 三输入OR门74LS30
5. 面包板
6. 示波器
7. 实验箱

二、实验内容及步骤

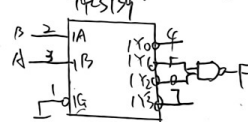
1. 测试74LS139中4选1数据选择器的逻辑功能



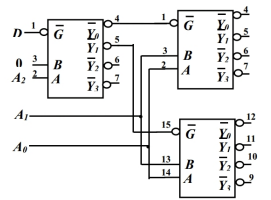
2. 测试74LS139中2-4译码器的逻辑功能



3. 用74LS139和74LS00实现逻辑函数 $F = A\bar{B} + \bar{A}B$

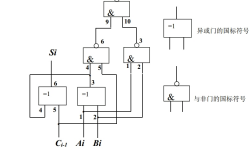


4. 用两片74LS139设计十进制数据分配器



5. 使用74LS00和74LS86实现全加器

$$S_i = A_i \oplus B_i \oplus C_{i-1} \quad C_i = (A_i \odot B_i) \vee (A_i \odot C_{i-1}) \vee (B_i \odot C_{i-1})$$



6. 用数据选择器74LS153设计全加器

$$S_i = \bar{A}_i \bar{B}_i C_{i-1} + \bar{A}_i B_i \bar{C}_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i B_i C_{i-1}$$

$$C_i = \bar{A}_i B_i C_{i-1} + A_i \bar{B}_i C_{i-1} + A_i B_i \bar{C}_{i-1} + A_i B_i C_{i-1}$$

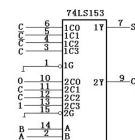
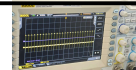
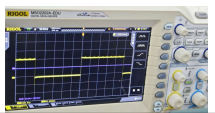


图 2.6 74LS153 实现全加器接线图

照片在下一页

三、数据分析及处理

1. 74LS153 功能:

输入		选通	输出
A	B	G	
0	0	1	
0	1	1	
1	0	1	
1	1	1	
0	0	0	
0	1	0	
1	0	0	
1	1	0	

2. 74LS139 功能:

(1代高亮 0代熄灭)

输入		选通	输出(LED)			
A	B	G	0	1	2	3
0	0	1	1	1	1	1
0	1	1	1	1	1	1
1	0	1	1	1	1	1
1	1	1	1	1	1	1
0	0	0	0	1	1	1
0	1	0	1	0	1	1
1	0	0	1	1	0	1
1	1	0	1	1	1	0

(1代高亮 0代熄灭)

输入		选通	输出(LED)
A	B	G	
0	0	1	1
0	1	1	1
1	0	1	1
1	1	1	1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	0	0

$$F = \bar{A}B + A\bar{B} = A \oplus B$$

四、实验结果讨论

- 实验收获:
 - ① 学习了 74LS153 与 74LS139 芯片各个引脚的功能与作用
 - ② 了解了数据选择器与译码器的作用
 - ③ 了解了全加器的作用
 - ④ 学习并掌握逻辑电路的设计方法
- 实验改进:
 - ① 实验芯片中没有非门, 故使用与非门来构造非门
 - ② 实验示波器无法显示高频率, 故降低频率实验

4.

(1代高亮 0代熄灭)

输入			输出(LED)							
A ₂	A ₁	A ₀	0	1	2	3	4	5	6	7
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0

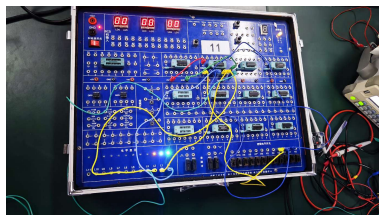
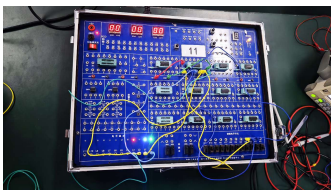
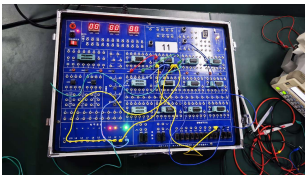
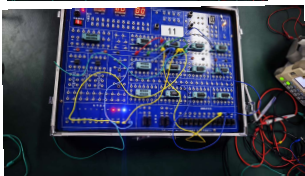
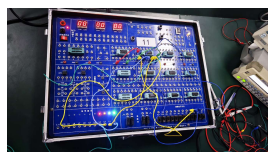
5-6.

(1代高亮 0代熄灭)

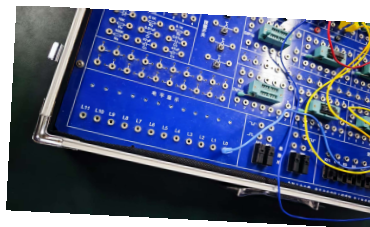
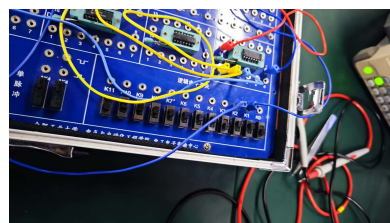
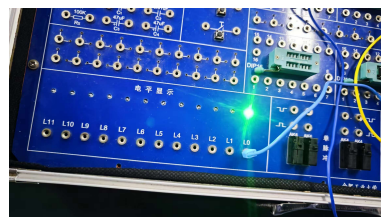
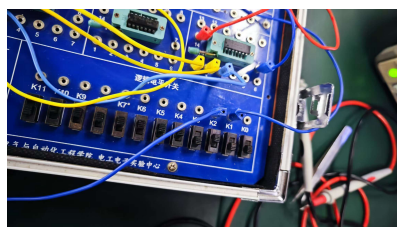
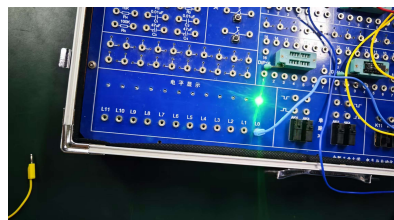
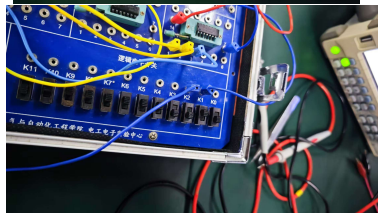
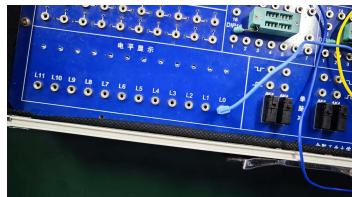
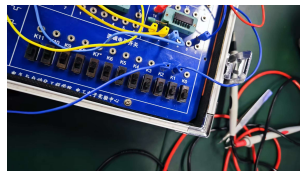
输入			输出(LED)	
A _i	B _i	C _{i-1}	LED1	LED0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



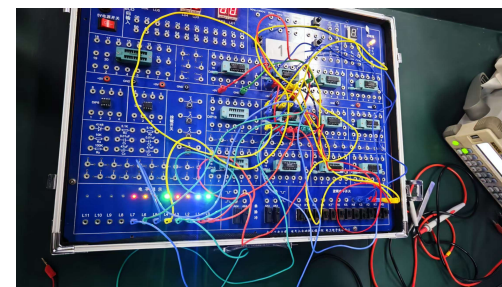
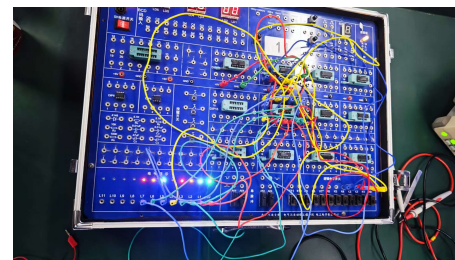
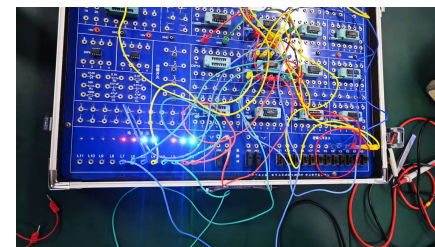
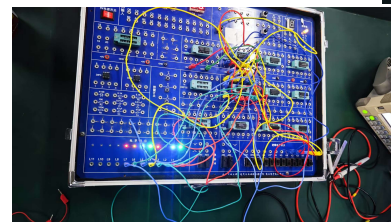
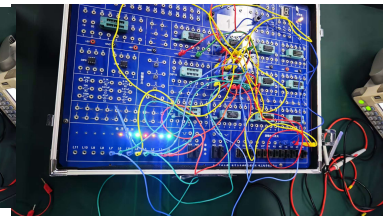
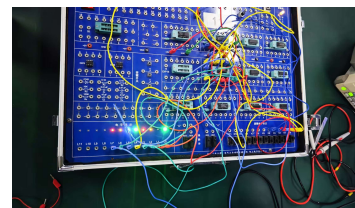
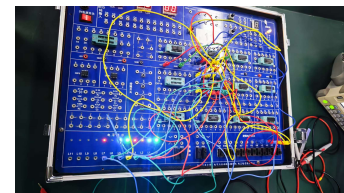
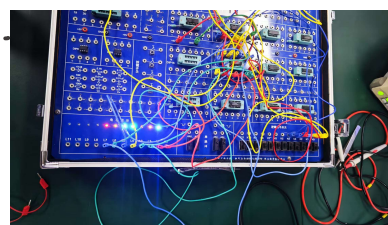
实验2:

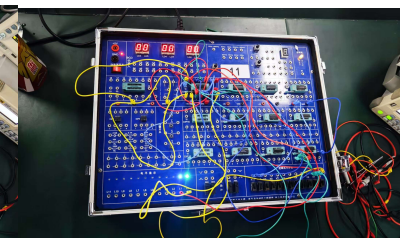
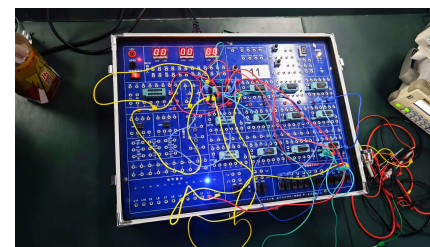
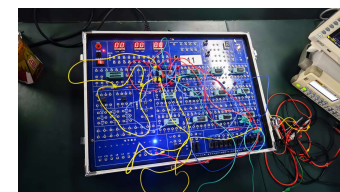
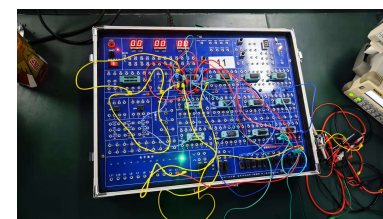
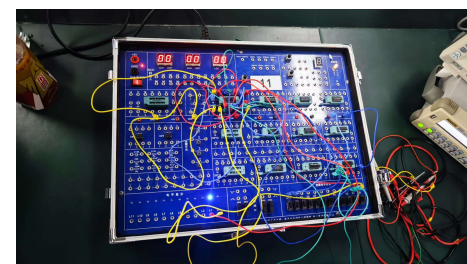
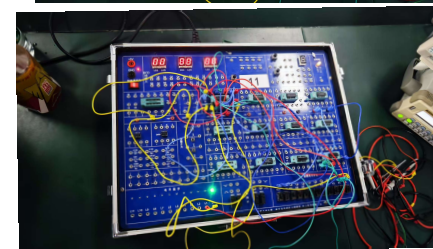
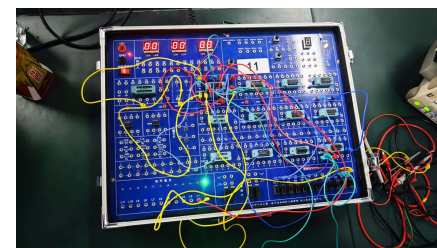
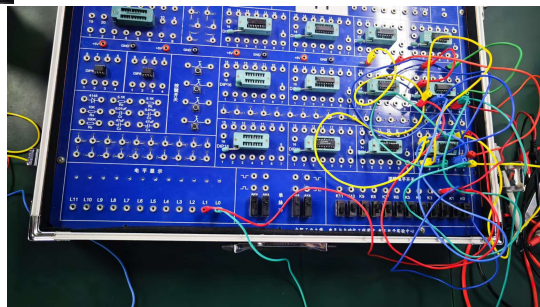
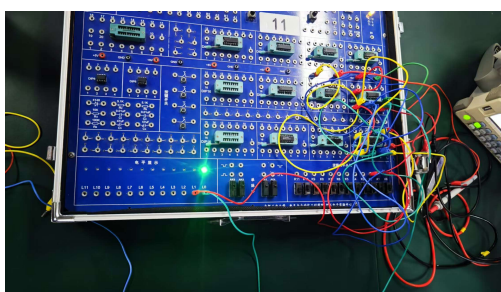
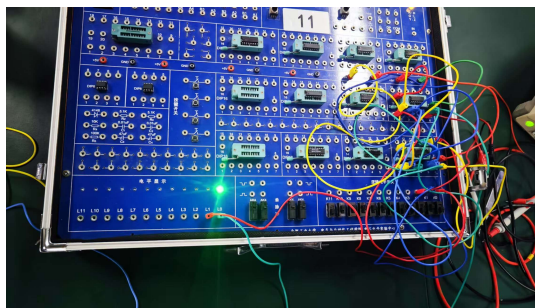
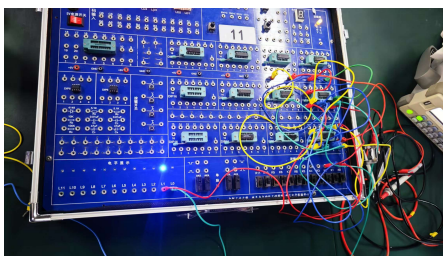
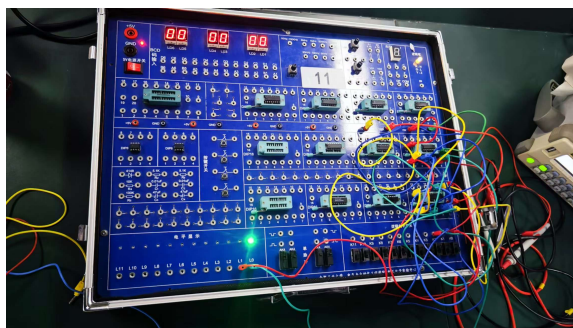
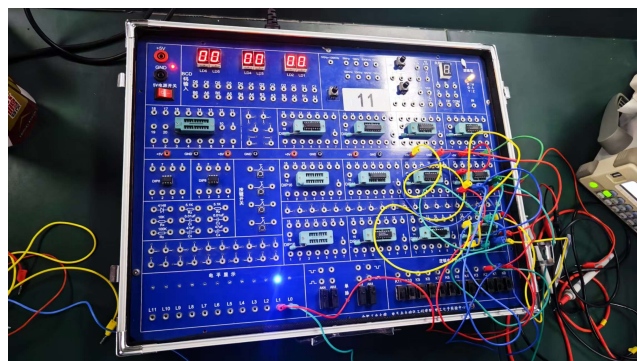
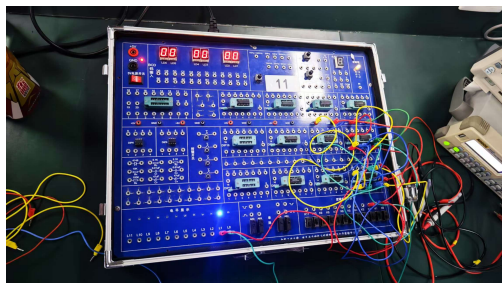


实验3:



实验4:





合肥工业大学实验预习报告

班级 计科23-3班 学号 2023210290 姓名 朱清扬 日期 2024.11.1

课程名称: 数字逻辑

实验项目: 触发器, 移位寄存器

- 实验目的
 - 掌握基本SR触发器、D触发器、JK触发器的工作原理
 - 学会正确正确使用SR触发器、D触发器、JK触发器
 - 熟悉移位寄存器电路结构及工作原理
 - 掌握中规模集成移位寄存器74LS194的运算功能及使用方法
- ### 二、实验原理

右图是基本SR触发器接线图。图中，K1、K2是电平开关输出，LED0、LED1是电平指示灯。基本SR

表 3.1 RS 锁存器真值表

输入		输出	
$\overline{R}$	$\overline{S}$	Q	$\overline{Q}$
0	0	1	1
0	1	1	0
1	0	0	1
1	1	Q	$\overline{Q}$

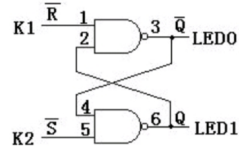


图 3.1 RS 锁存器

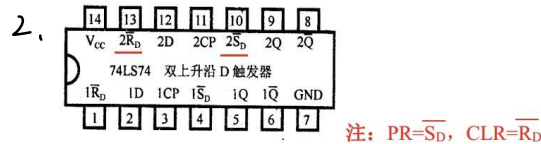


图 3.2 74LS74 引脚图

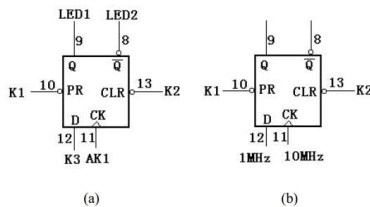


图 3.3 D 触发器

上图是测试D触发器的接线图，K1、K2、K3是电平开关输出，LED0、LED1是电平指示灯，AK1是宽单脉冲，1MHz、10MHz是时钟脉冲。左图为单脉冲的测试，右图为连续脉冲的测试。

表 3.2 D 触发器输出

输入				输出	
PR	CLR	CLK	D	Q	$\overline{Q}$
L	H	X	X	H	L
H	L	X	X	L	H
H	H	↑	H	H	L
H	H	↑	L	L	H
H	H	L	X	Q	$\overline{Q}$

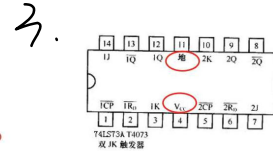


图 3.4 74LS73A 芯片引脚图

上图是测试JK触发器的接线图，K2、K3、K4是电平开关输出，LED0、LED1是电平指示灯，AK1是宽单脉冲，74LS73引脚4接+5V，引脚11接地，74LS73只有复位端CLR。

表 3.2 JK 触发器输出

输入				输出	
清除	时钟	J	K	Q	$\overline{Q}$
L	X	X	X	L	H
H	↓	L	L	Q ₀	$\overline{Q}_0$
H	↓	L	H	L	H
H	↓	H	L	H	L
H	↓	H	H	反相	反相
H	H	X	X	Q ₀	$\overline{Q}_0$

合肥工业大学实验报告

开课实验室 A402 成绩 指导老师 刘军

课程名称: 数字逻辑

实验项目: 触发器, 移位寄存器

- ### 一、实验仪器
- 面包板上集成芯片 74LS00
 - 双D触发器 74LS74
 - 12位双向通用移位寄存器 74LS194
 - 双JK触发器 74LS73
 - 万用表
 - 示波器
 - 实验箱

- ### 二、实验内容及步骤
- 设计基本SR触发器并验证功能
 - 验证D触发器
 - 验证JK触发器
 - 使用74LS74双D触发器构成右移寄存器
 - 使用74LS74双D触发器构成双向移位寄存器
 - 验证双向移位寄存器74LS194的运算功能

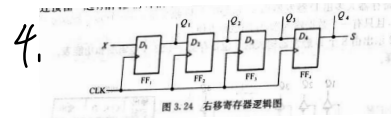


图 3.24 右移寄存器逻辑图

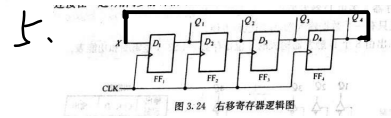


图 3.24 右移寄存器逻辑图

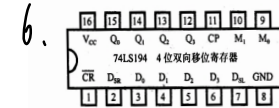


图 3.6 74LS194 芯片引脚图

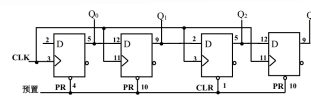
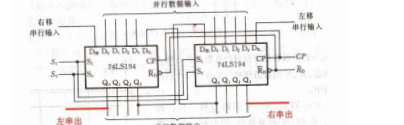


图 3.7 74LS194 内部图



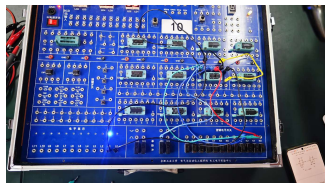
输入				输出				功能
/CR	M1	M0	CP	DSL	DSR	ABCD	Q4 Q3 Q2 Q1	
L	X	X	X	X	X		L L L L	清零
H	H	H	↑	X	X	a b c d	a b c d	置数
H	L	H	↑	X	H		H Q4 Q3 Q2	右移
H	L	H	↑	X	L		L Q4 Q3 Q2	左移
H	H	L	↑	X	H		Q4 Q3 Q2 H	右移
H	H	L	↑	X	L		Q4 Q3 Q2 L	左移
H	L	L	X	X	X		Q4 Q3 Q2 Q1	保持



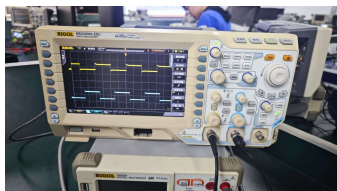
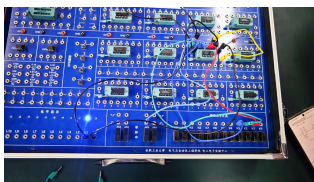
三、数据分析及处理

其中4.5.6为设计性实验电路连线图

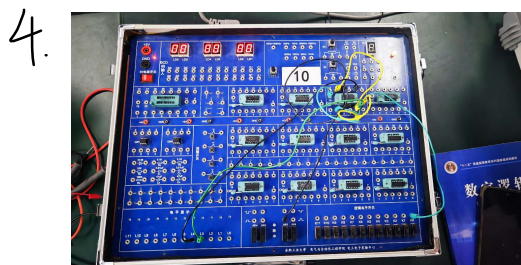
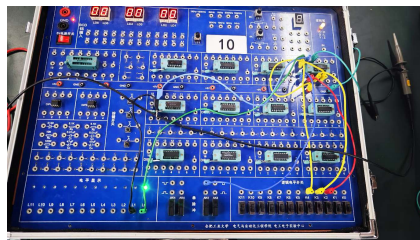
1. (1) $\overline{R}=0, \overline{S}=1$, 测得 $\overline{Q}=1, Q=0$ 。
- (2) $\overline{R}=1, \overline{S}=1$, 测得 $\overline{Q}=1, Q=0$ 。
- (3) $\overline{R}=1, \overline{S}=0$, 测得 $\overline{Q}=0, Q=1$ 。
- (4) $\overline{R}=1, \overline{S}=1$, 测得 $\overline{Q}=0, Q=1$ 。
- (5) $\overline{R}=0, \overline{S}=0$, 测得 $\overline{Q}=1, Q=0$ 。



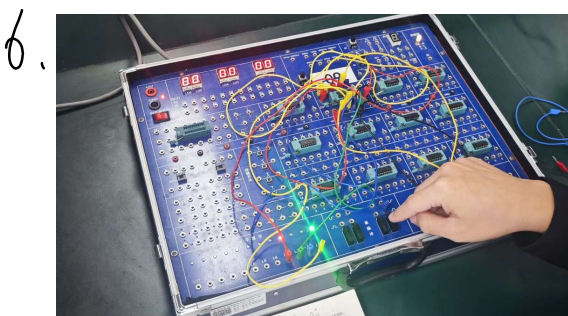
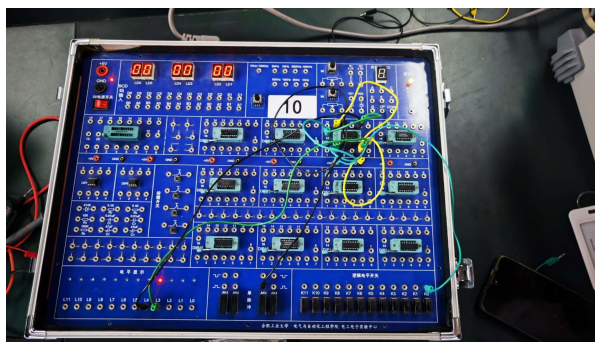
2. (1) $\text{CLR}=0, \text{PR}=1$, 测得 $\overline{Q}=1, Q=0$ 。
- (2) $\text{CLR}=1, \text{PR}=1$, 测得 $\overline{Q}=1, Q=0$ 。
- (3) $\text{CLR}=1, \text{PR}=0$, 测得 $\overline{Q}=0, Q=1$ 。
- (4) $\text{CLR}=1, \text{PR}=1$, 测得 $\overline{Q}=0, Q=1$ 。
- (5) $\text{CLR}=1, \text{PR}=1, D=1$, CK 接宽单脉冲, 按按钮, 测得 $\overline{Q}=0, Q=1$ 。
- (6) $\text{CLR}=1, \text{PR}=1, D=0$, CK 接宽单脉冲, 按按钮, 测得 $\overline{Q}=1, Q=0$ 。
- (7) $\text{CLR}=1, \text{PR}=1, D$ 接 1MHz 脉冲, CK 接 10MHz, 在示波器上同时观测 Q、CK



3. (1) $\text{CLR}=0$, 测得 $\overline{Q}=1, Q=0$ 。
- (2) $\text{CLR}=1, J=0, K=0$, 按宽单脉冲按钮 AK1, 测得 $\overline{Q}=1, Q=0$ 。
- (3) $\text{CLR}=1, J=1, L=0$, 按宽单脉冲按钮 AK1, 测得 $\overline{Q}=0, Q=1$ 。
- (4) $\text{CLR}=1, J=0, K=0$, 按宽单脉冲按钮 AK1, 测得 $\overline{Q}=0, Q=1$ 。
- (5) $\text{CLR}=1, J=0, K=1$, 按宽单脉冲按钮 AK1, 测得 $\overline{Q}=1, Q=0$ 。
- (6) $\text{CLR}=1, J=0, K=0$, 按宽单脉冲按钮 AK1, 测得 $\overline{Q}=1, Q=0$ 。
- (7) $\text{CLR}=1, J=1, K=1$, 按宽单脉冲按钮 AK1, 测得 $\overline{Q}=0, Q=1$; 再按宽单脉冲按钮 AK1, 测得 $\overline{Q}=1, Q=0$ 。



5.



四、实验结果讨论

收获: 1. 了解 SR 触发器内部构造

2. 了解 SR、JK 触发器工作原理与真值表

3. 学会用 D 触发器设计移位寄存器

4. 通过实验了解到所有触发器在电路中的具体应用

不足与改进: 1. 实验中 LED 灯有故障, 存在闪烁现象

2. 构造移位寄存器时由于线材不够, 只构造了二位移位, 效果不明显。

遇到问题的改进: 1. 用手波发生器时波形不明显

2. 构造移位寄存器时无灯亮

应光预置输入后再进行移位, 能正确现象



合肥工业大学实验预习报告

班级 计科23-3班 学号 20230229 姓名 李新阳 日期 2024.11.5

课程名称: 数字逻辑

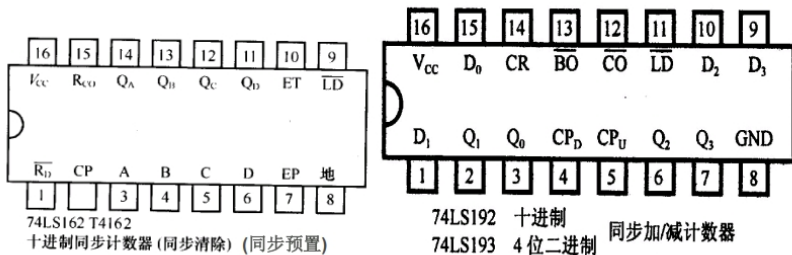
实验项目: 《计数器》

一、实验目的

- 熟悉74LS162和74LS193的逻辑功能
- 熟悉计数器的工作原理和方法
- 设计计数器如课程例
- 掌握计数器的实验方法

二、实验原理

5. 学习用中规模集成电路的设计方法



1. 复位法构成的模7计数器接线图

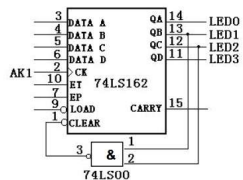


图 4.1 复位法7进制计数器接线图

图中, AK1 是按单脉冲按钮 AK1 产生的单脉冲, LED0、LED1、LED2 和 LED3 是电平指示灯, 1MHz 是实验台上的时钟脉冲源。

2. 预置法模7计数器接线图

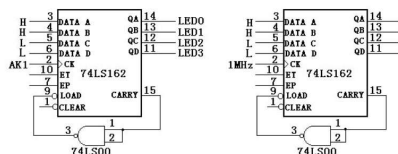


图 4.2 预置法7进制计数器接线图

图中, AK1 是按单脉冲按钮 AK1 产生的单脉冲, LED0、LED1、LED2 和 LED3 是电平指示灯图中, H、L 分别为高电平、低电平, 1MHz 是计数脉冲源。

合肥工业大学实验报告

开课实验室 A402 成绩 指导老师 刘洋

课程名称: 数字逻辑

实验项目: 《计数器》

一、实验仪器

1. 同步计数器 74LS193, 异步清除, 异步预置, 四位二进制可逆计数器
2. 2片同步十进制加法计数器 74LS162
3. 1片 74LS00
4. 实验箱

二、实验内容及步骤

1. 用1片74LS162和1片74LS00采用复位法构成一个模7计数器。用单脉冲按钮时钟, 观测计数状态, 并记录。
2. 用1片74LS162和1片74LS00采用预置法构成一个模7计数器。用单脉冲按钮时钟, 观测计数状态, 并记录。用 1MHz 时钟计数时钟, 并记录。
3. 用1片74LS162和1片74LS00构成模60计数器。用单脉冲按钮时钟, 观测计数管数变化。
4. 用1片74LS193采用预置法构成一个模8计数器。
5. 用1片74LS193采用预置法构成一个模8计数器。

4. 复位法构成模8计数器

1. 同预置法接线, 在7(111)时使位
2. 预置法构成模8计数器
同预置法2接线, 预置数设为8

74LS193是低电平有效
不用构成非门

3. 模60计数器接线图

(1) 预置法模60计数器接线图

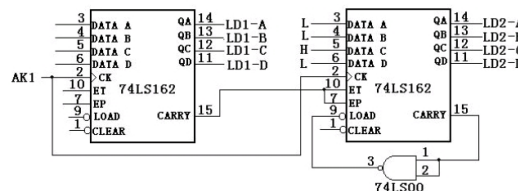


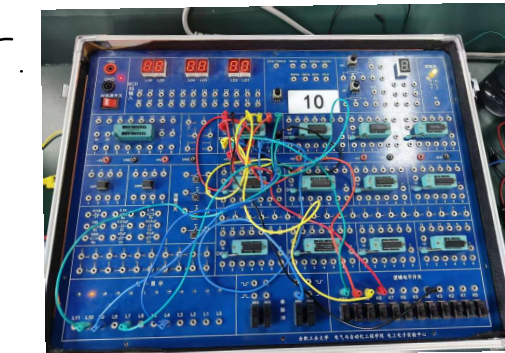
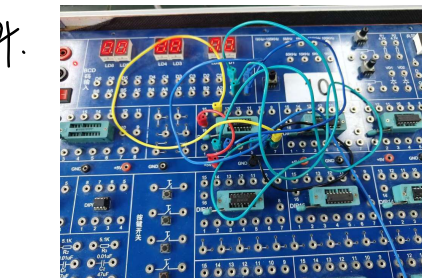
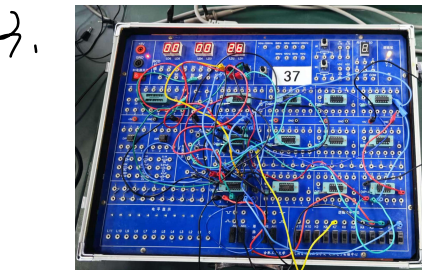
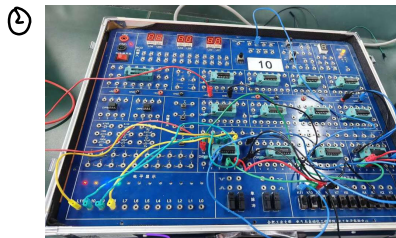
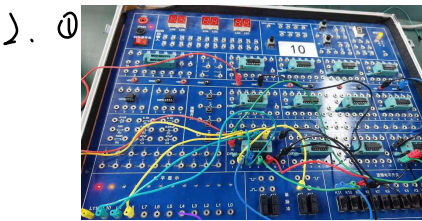
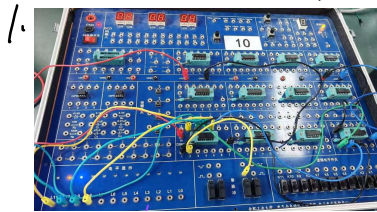
图 4.3 预置法模60计数器接线图

图中, LED1-A, LED1-B, LED1-C, LED1-D, LED2-A, LED2-B, LED2-C, LED2-D 是数码管的数据端, AK1 是按单脉冲按钮 AK1 产生的单脉冲, H、L 是高低电平。



三、数据分析及处理

本次实验数据表格, 故不贴接线图(设计台图片)



四、实验结果讨论

- 收获:
1. 实验中了解了计数器各个引脚的功能
 2. 明白了如何利用复位信号与置数法来构建任意模数计数器
 3. 观察到了自启动现象
 4. 加深了对计数器的理解以及计数器的设计

不足与改进: 1. 实验中连线存在接角不良的现象, 导致LED灯显示存在一定问题。

2. 在设计计数器时使用LED灯来显示不如数码管直接, 遇到困难能够解决。

1. 实验中芯片工作不正常, 没有将使能端接高电平, 误认为是芯片故障。

2. 实验构建计数器产生错误循环, 没有仔细查看有效电平, 两个芯片有效电平不一样, 导致有效电平异常。



一. 练习内容

1. 用 Verilog 完成三态门的设计，并用 Vivado 进行仿真。
2. 用 Verilog 完成 4 位加法器的设计，并使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 三态门设计

设计代码：

```
`timescale 1ns / 1ps

module tri_state_buffer (
    input wire in,
    input wire enable,
    output wire out
);

    assign out = (enable) ? in : 1'bz;

endmodule
```

测试代码：

```
`timescale 1ns / 1ps

module test;

    reg in;
    reg enable;
    wire out;

    san_tai_men uut (
        .in(in),
        .enable(enable),
        .out(out)
    );

endmodule
```

```

initial begin
    in = 0;
    enable = 0;
    #10;

    in = 1;
    enable = 0;
    #10;

    in = 1;
    enable = 1;
    #10;

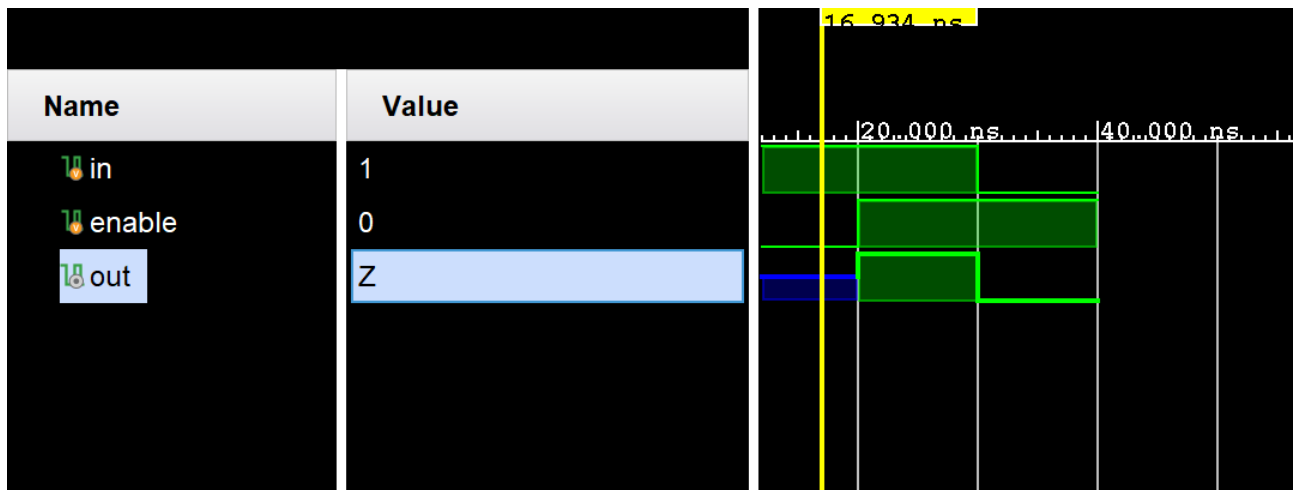
    in = 0;
    enable = 1;
    #10;

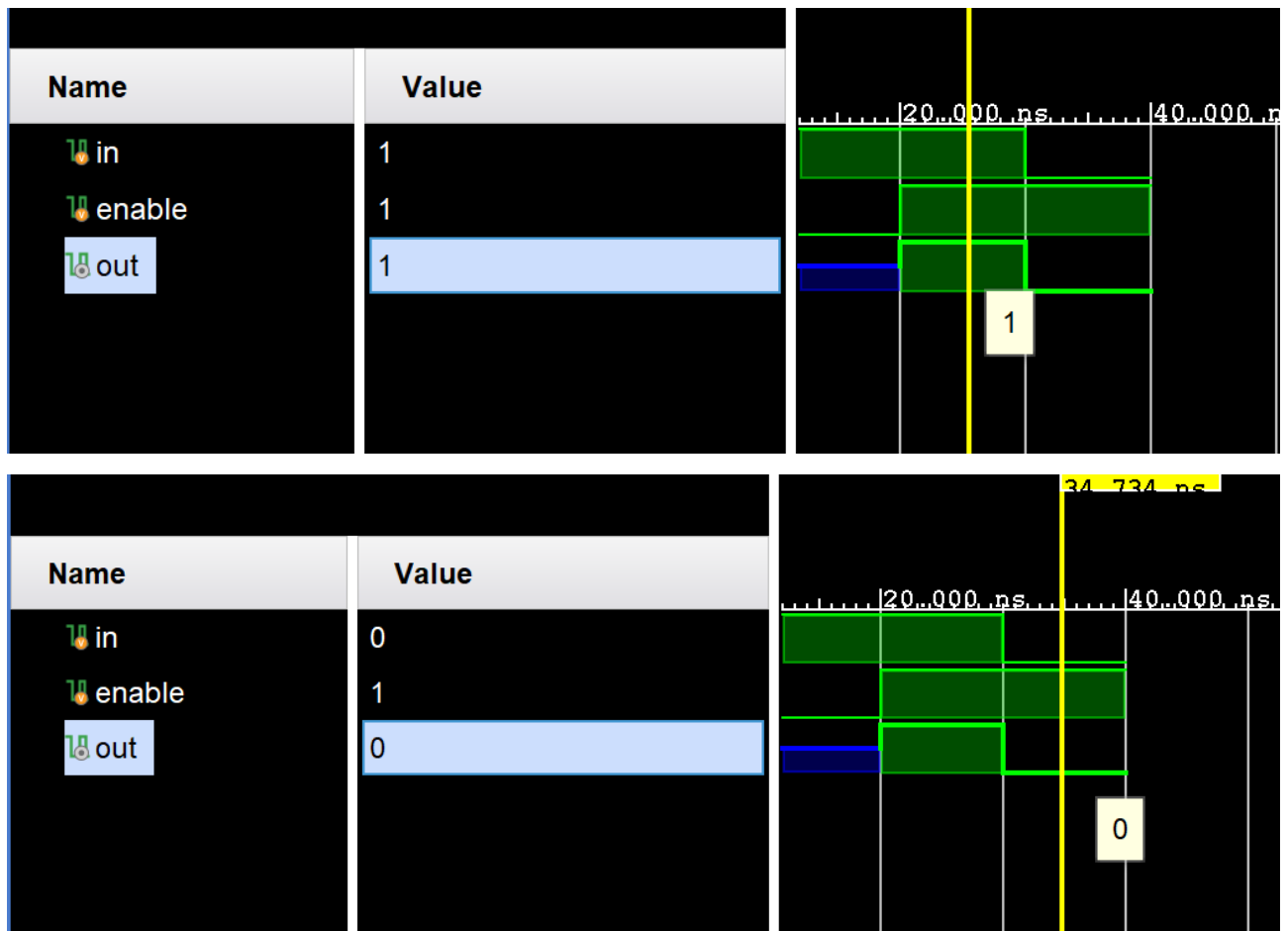
    $finish;
end

endmodule

```

仿真图：





2.4 位加法器的设计

设计代码:

```

`timescale 1ns / 1ps
module quan_jia_qi(
    cin,
    a,
    b,
    sum,
    cout
);
    input cin;
    input [3:0] a, b;

    output cout;
    output [3:0] sum;

    assign {cout, sum} = a + b + cin;
endmodule

```

测试代码:

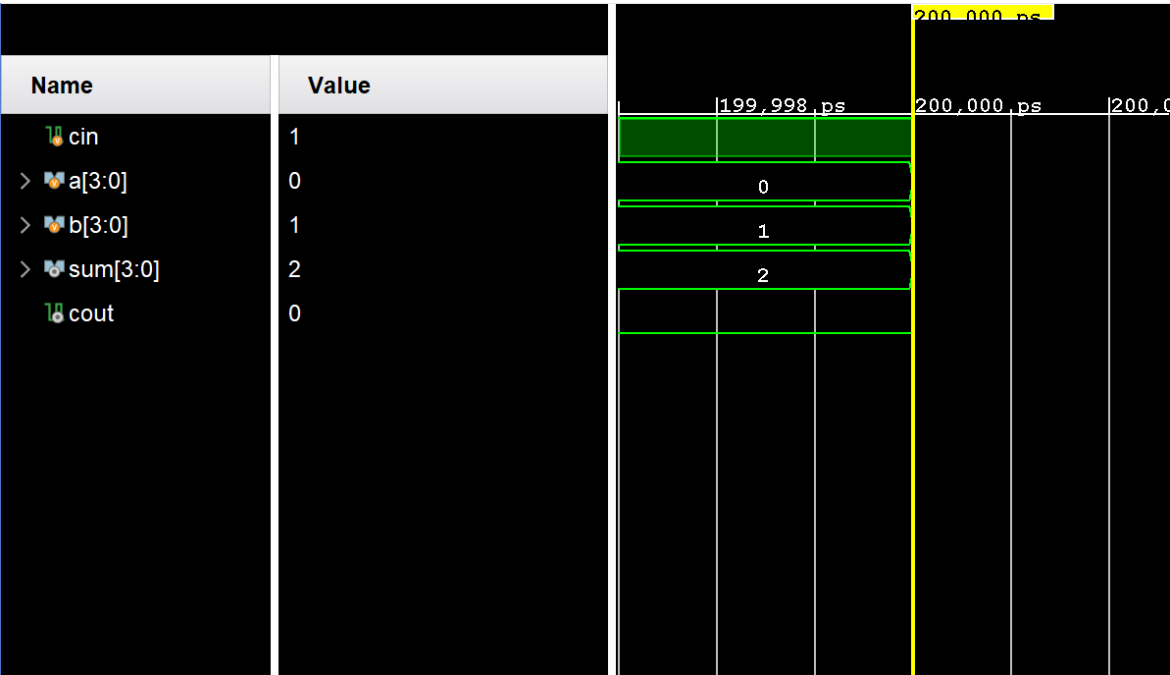
```
`timescale 1ns / 1ps
module test;

    reg cin;
    reg [3:0] a, b;
    wire [3:0] sum;
    wire cout;

    quan_jia_qi qjq(
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        a = 0;
        b = 0;
        cin = 0;
        #100;
        a = 0;
        b = 1;
        cin = 1;
        #100;
        $finish;
    end
endmodule
```


仿真图：



一. 练习内容

1. 用 Verilog 完成 3-8 译码器的设计，并用 Vivado 进行仿真。
2. 用 Verilog 完成两个 4 位二进制数据比较器的设计，并使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 3-8 译码器的设计（无效输出低电平）:

设计代码:

```
`timescale 1ns / 1ps

module yi_ma_3_to_8 (
    input [2:0] a,
    input enable,
    output reg [7:0] y
);

    always @(*) begin
        if (enable) begin
            case (a)
                3'b000: y = 8'b0000_0001;
                3'b001: y = 8'b0000_0010;
                3'b010: y = 8'b0000_0100;
                3'b011: y = 8'b0000_1000;
                3'b100: y = 8'b0001_0000;
                3'b101: y = 8'b0010_0000;
                3'b110: y = 8'b0100_0000;
                3'b111: y = 8'b1000_0000;
                default: y = 8'b0000_0000;
            endcase
        end else begin
            y = 8'b0000_0000;
        end
    end
end

endmodule
```

测试代码：

```
`timescale 1ns / 1ps

module test;

    reg [2:0] a;
    reg enable;
    wire [7:0] y;

    yi_ma_3_to_8 uut (
        .a(a),
        .enable(enable),
        .y(y)
    );

    initial begin
        a = 3'b000;
        enable = 0;
        #10;

        a = 3'b000;
        enable = 1;
        #10;

        a = 3'b001;
        enable = 1;
        #10;

        a = 3'b010;
        enable = 1;
        #10;

        a = 3'b011;
        enable = 1;
        #10;

        a = 3'b100;
        enable = 1;
        #10;

        a = 3'b101;
        enable = 1;
```



```

#10;

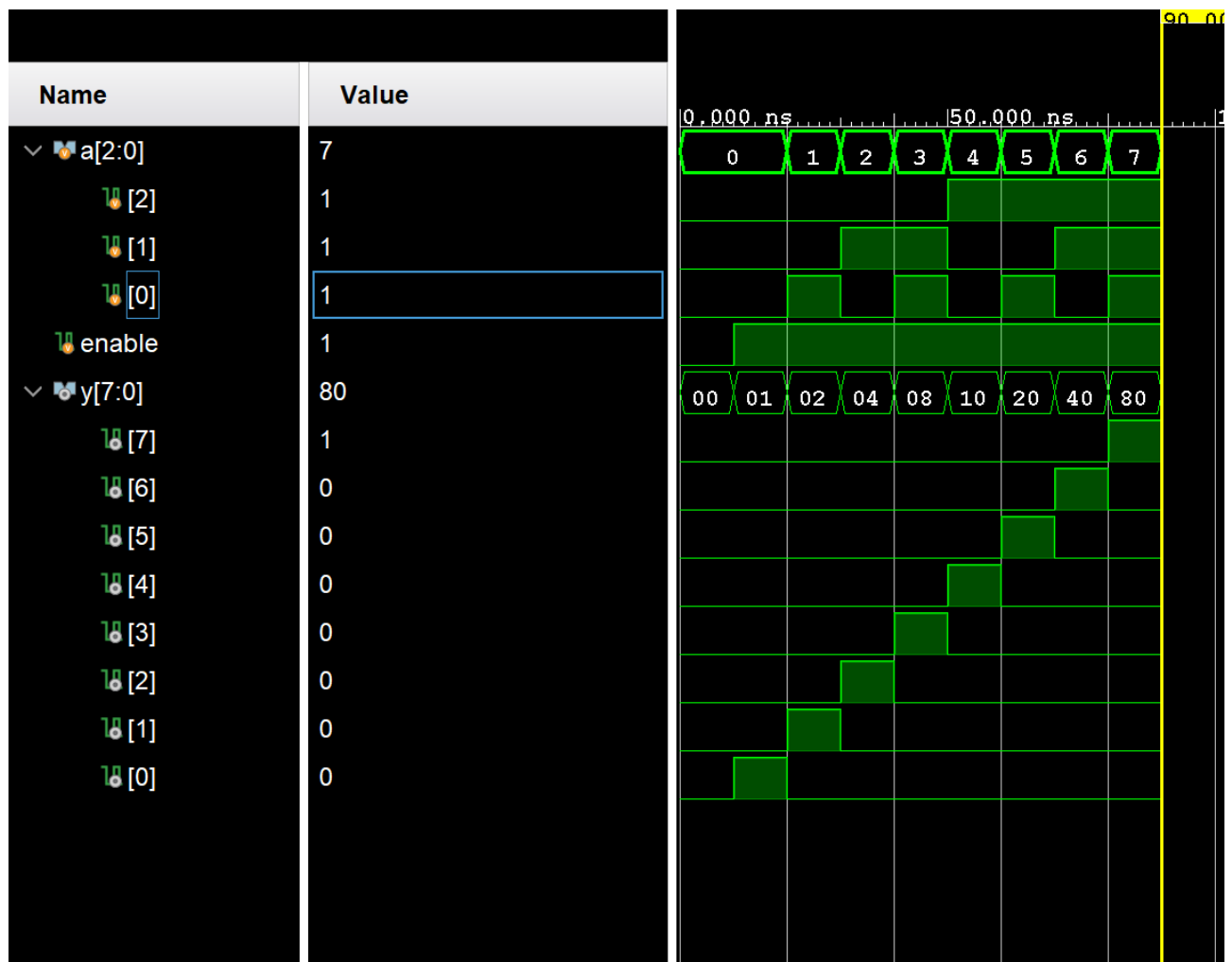
a = 3'b110;
enable = 1;
#10;

a = 3'b111;
enable = 1;
#10;

$finish;
end
endmodule

```

仿真图：



2. 四位二进制数据比较器

设计代码:

```
`timescale 1ns / 1ps

module binary_comparator (
    input [3:0] a,
    input [3:0] b,
    output reg greater,
    output reg equal,
    output reg less
);

always @(*) begin
    if (a > b) begin
        greater = 1;
        equal = 0;
        less = 0;
    end else if (a == b) begin
        greater = 0;
        equal = 1;
        less = 0;
    end else begin
        greater = 0;
        equal = 0;
        less = 1;
    end
end

endmodule
```

测试代码:

```
`timescale 1ns / 1ps

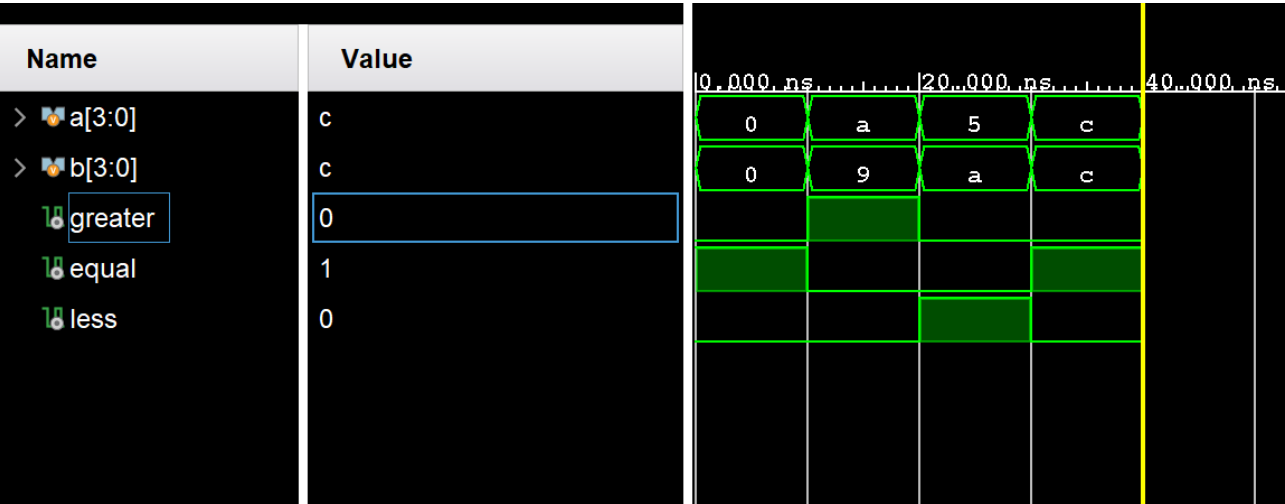
module tb_binary_comparator;

    reg [3:0] a;
    reg [3:0] b;
    wire greater;
    wire equal;
    wire less;
```

```
binary_comparator uut (  
    .a(a),  
    .b(b),  
    .greater(greater),  
    .equal(equal),  
    .less(less)  
);
```

```
initial begin  
    a = 4'b0000;  
    b = 4'b0000;  
  
    #10;  
  
    a = 4'b1010;  
    b = 4'b1001;  
    #10;  
  
    a = 4'b0111;  
    b = 4'b1010;  
    #10;  
  
    a = 4'b1000;  
    b = 4'b1000;  
    #10;  
  
    $finish;  
end  
  
endmodule
```


仿真图：



一. 练习内容

1. 用 Verilog 完成具有异步清零和异步置 1 的 D 触发器的设计,并用 Vivado 进行仿真。
2. 用 Verilog 完成时钟下降沿触发的 JK 触发器的设计, 使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 具有异步清零和异步置 1 的 D 触发器。

设计代码:

```
`timescale 1ns / 1ps

module d (
    input clk,
    input rst_n,
    input set_n,
    input d,
    output reg q
);

always @(posedge clk or negedge rst_n or negedge set_n) begin
    if (!rst_n) begin
        q <= 1'b0;
    end else if (!set_n) begin
        q <= 1'b1;
    end else begin
        q <= d;
    end
end

endmodule
```

测试代码:

```
`timescale 1ns / 1ps

module test;
```

```
reg clk;
reg rst_n;
reg set_n;
reg d;
wire q;

d uut (
    .clk(clk),
    .rst_n(rst_n),
    .set_n(set_n),
    .d(d),
    .q(q)
);

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
```

```
initial begin
    rst_n = 1;
    set_n = 1;
    d = 0;
    #10;

    rst_n = 0;
    #10;

    rst_n = 1;
    #10;

    set_n = 0;
    #10;

    set_n = 1;
    #10;

    d = 1;
    #100;

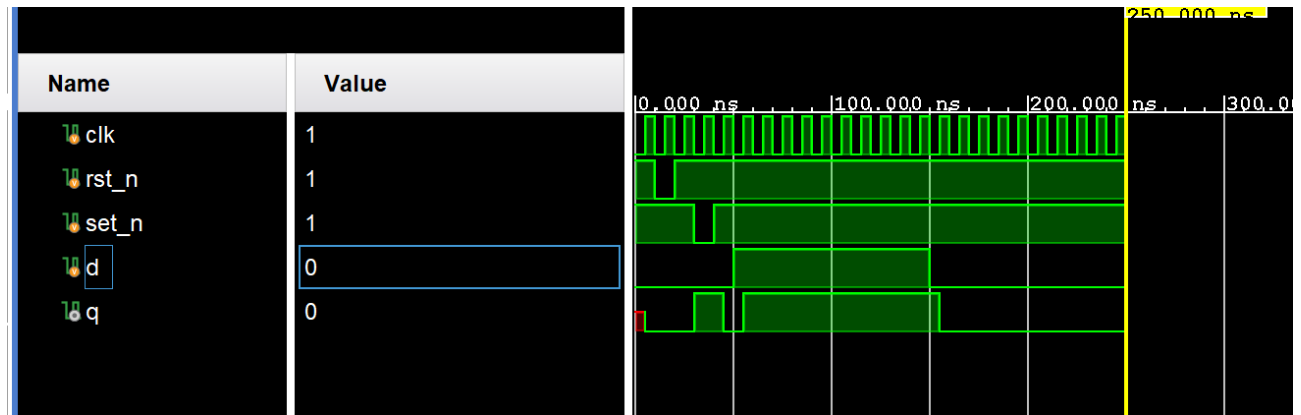
    d = 0;
    #100;

    $finish;
```

end

endmodule

仿真图：



2. 时钟下降沿触发的 JK 触发器

设计代码：

```
`timescale 1ns / 1ps
```

```
module jk (  
    input clk,  
    input rst_n,  
    input j,  
    input k,  
    output reg q,  
    output reg q_n  
);  
  
always @(negedge clk or negedge rst_n) begin  
    if (!rst_n) begin  
        q <= 1'b0;  
        q_n <= 1'b1;  
    end else begin  
        if (j == 1 && k == 1) begin  
            q <= ~q;  
            q_n <= ~q_n;  
        end else if (j == 1 && k == 0) begin  
            q <= 1'b1;  
            q_n <= 1'b0;  
        end  
    end  
end
```



```

        end else if (j == 0 && k == 1) begin
            q <= 1'b0;
            q_n <= 1'b1;
        end else begin
            end
        end
    end
end

endmodule

```

测试代码:

```
`timescale 1ns / 1ps
```

```
module test;
```

```

reg clk;
reg rst_n;
reg j;
reg k;
wire q;
wire q_n;

```

```

jk uut (
    .clk(clk),
    .rst_n(rst_n),
    .j(j),
    .k(k),
    .q(q),
    .q_n(q_n)
);

```

```

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

```

```

initial begin
    rst_n = 1;
    j = 0;
    k = 0;
    #10;

    rst_n = 0;

```

```

#10;

rst_n = 1;
#10;

j = 1;
k = 0;
#20;

j = 0;
k = 1;
#20;

j = 1;
k = 1;
#20;

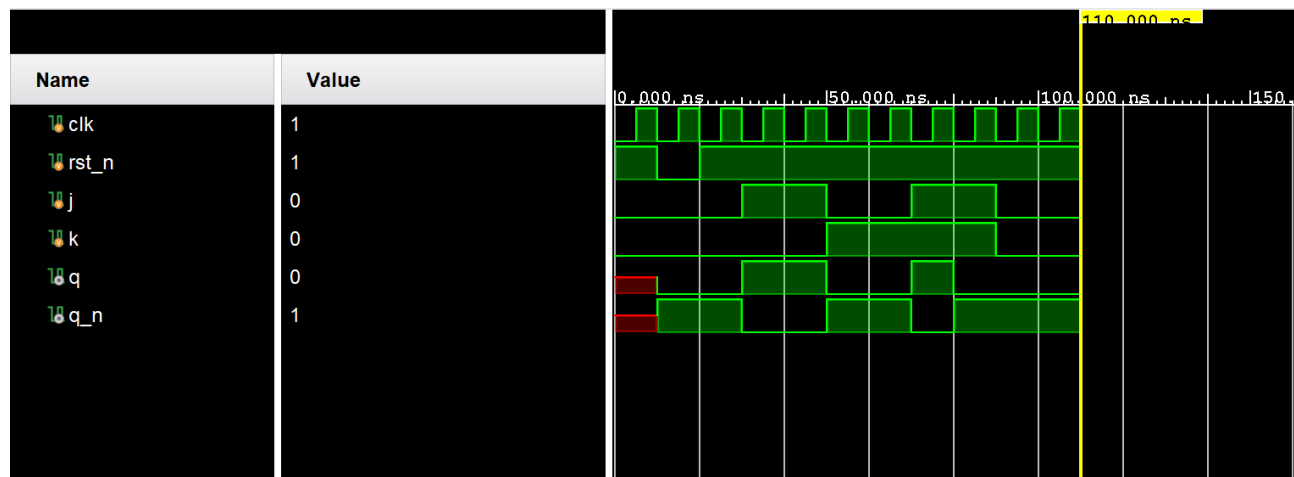
j = 0;
k = 0;
#20;

$finish;
end

endmodule

```

测试图：



一. 练习内容

1. 用 Verilog 完成具有异步清零和异步置 1 的模 10 计数器的设计，并用 Vivado 进行仿真。
2. 用 Verilog 完成具有同步清零和同步置 1 的模 16 计数器的设计，使用 Vivado 进行仿真。

二. 源程序及仿真结果截图

1. 有异步清零和异步置 1 的模 10 计数器（低电平有效）

设计代码：

```
`timescale 1ns / 1ps

module mod10_counter (
    input clk,
    input rst_n,
    input set_n,
    output reg [3:0] count
);

// Asynchronous reset and set
always @(posedge clk or negedge rst_n or negedge set_n) begin
    if (!rst_n) begin
        count <= 4'b0000;
    end else if (!set_n) begin
        count <= 4'b1001;
    end else begin
        if (count == 4'b1001) begin
            count <= 4'b0000;
        end else begin
            count <= count + 1;
        end
    end
end

endmodule
```

测试代码:

```
`timescale 1ns / 1ps
```

```
module test;
```

```
reg clk;
```

```
reg rst_n;
```

```
reg set_n;
```

```
wire [3:0] count;
```

```
mod10_counter uut (
```

```
    .clk(clk),
```

```
    .rst_n(rst_n),
```

```
    .set_n(set_n),
```

```
    .count(count)
```

```
);
```

```
initial begin
```

```
    clk = 0;
```

```
    forever #5 clk = ~clk;
```

```
end
```

```
initial begin
```

```
    rst_n = 1;
```

```
    set_n = 1;
```

```
    #10;
```

```
    rst_n = 0;
```

```
    #10;
```

```
    rst_n = 1;
```

```
    #10;
```

```
    set_n = 0;
```

```
    #10;
```

```
    set_n = 1;
```

```
    #10;
```

```
    #100;
```

```
    #50;
```

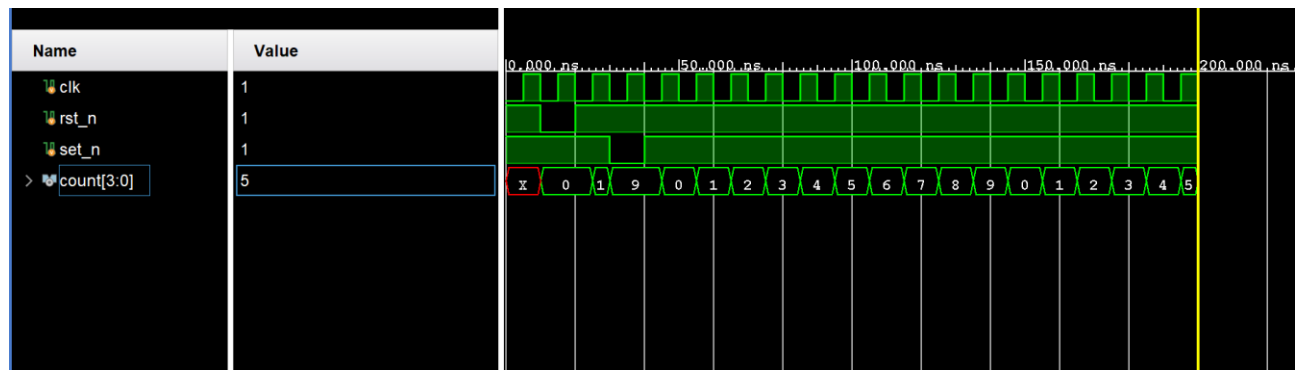
```

    $finish;
end

endmodule

```

仿真图：



2. 同步清零和同步步置 1 的模 16 计数器

设计代码：

```

`timescale 1ns / 1ps

module mod16_counter (
    input wire clk,
    input wire clr,
    input wire load,
    input wire [3:0] data,
    output reg [3:0] count
);

always @(posedge clk) begin
    if (clr) begin
        count <= 4'b0000;
    end else if (load) begin
        count <= data;
    end else begin
        if (count == 4'b1111) begin
            count <= 4'b0000;
        end else begin
            count <= count + 1;
        end
    end
end

```



```
        end
    end

endmodule
```

测试代码:

```
`timescale 1ns / 1ps

module test;

    reg clk;
    reg clr;
    reg load;
    reg [3:0] data;
    wire [3:0] count;

    mod16_counter uut (
        .clk(clk),
        .clr(clr),
        .load(load),
        .data(data),
        .count(count)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        clr = 0;
        load = 0;
        data = 4'b0000;

        #10;

        clr = 1;
        #10;

        clr = 0;
        #10;

        data = 4'b1010;
```

```

load = 1;
#10;

load = 0;
#10;

#100;

#50;

clr = 1;
#10;

clr = 0;
#10;

data = 4'b1100;
load = 1;
#10;

load = 0;
#10;

$finish;
end

endmodule

```

仿真图：

